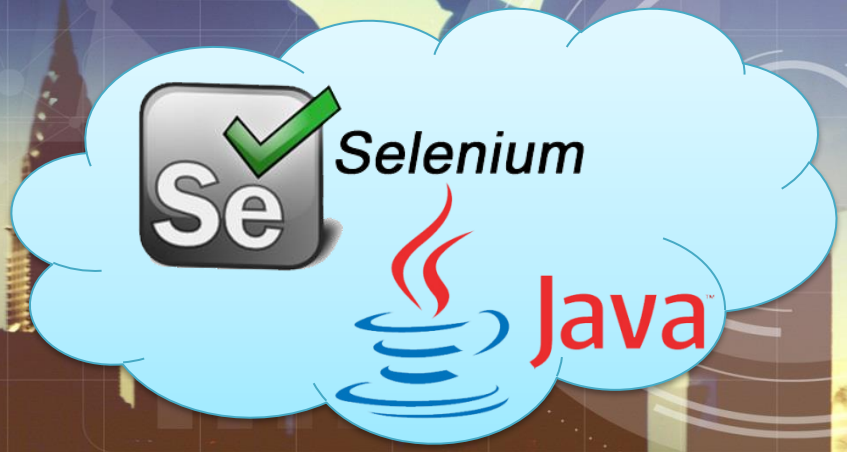
Waits in WebDriver
WebDriver - Lecture - 4



Agenda

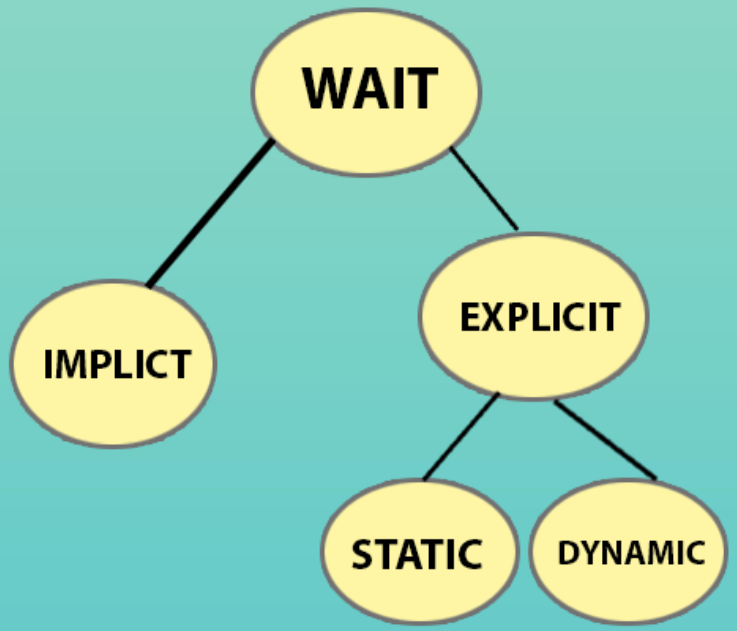
- Waits in WebDriver
- Implicit Wait
- NoSuchElementException
- Sub-Interfaces of WebDriver
- Explicit Wait
- “until()” method in Fluent Wait
- ExpectedConditions Class
- Explicit Methods Returning WebElement
- Verify Page Title Explicitly
- Quiz
- Homework

Waits in WebDriver

- In WebDriver, **"Waits"** play an important role in executing tests because:
  - When a page is loaded by the browser, the elements which we want to interact with, may load at different **time intervals**.
  - **Ajax and JavaScript** are more prominent in web pages.
  - **Web-Element** may be displayed based on certain **condition**, when a web-page is updated without reloading the whole page again.

- Until now, we were using static wait **Thread.Sleep()** which it is not recommended to use

- WebDriver API, provides us with two waits:
  - **Implicit Wait**
  - **Explicit Wait (Dynamic)**



Implicit Wait

```
driver.manage().timeouts().implicitlyWait(TimeOut, TimeUnit.SECONDS);
```

- The concept of **implicit wait** originally came from **WATIR (Web Application Testing in Ruby)**

- The **implicit wait**:
  - Once applied, it is applicable for all the elements found with **WebDriver's find() method** till the end of the script
  - Specifies the amount of time, the **driver** should wait when **searching for an element** if it is **not immediately present**
  - When searching for an element, the **driver** poll (**look in**) the page until the element has been found, or the timeout expires before throwing a **"NoSuchElementException"**
  - The default setting **for timeout is zero**.
    - Once we set the time, **WebDriver** will wait for the **timeout** duration before throwing an **exception**

- **Implicit wait** will accept 2 parameters:
  - The first parameter will accept the **timeout** as an integer value
  - Second parameter will accept the **time measurement** in terms of **seconds, minutes, milliseconds, microseconds etc.**

- **Implicit wait** is not of much use and remains only as a **decorative line** in the script because:
  - It is applied without any condition.

- Increasing the **implicit wait timeout**, will **adversely** increase test run time

NoSuchElementException

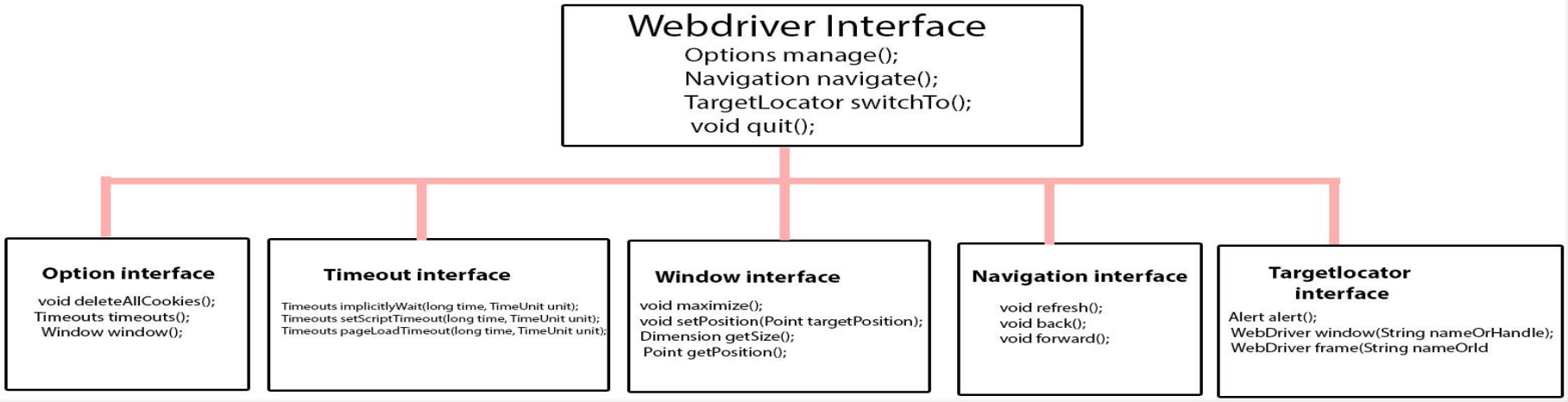
- **WebDriver** throws **org.openqa.selenium.NoSuchElement Exception**, when web-element is not found on **page DOM**.
- This exception extends **RuntimeException** class which is an **unchecked** exception.
- **Implicit wait** only delays this exception with the **timeout** duration and does not find the element on the basis of some condition whereas **Explicit wait** works with a condition**.**

```
driver.findElement(By.xpath("//*[@id='xyz']")).click();
```

Invalid XPath locator, using which no element on the Web Page can be located

```
Exception in thread "main" org.openqa.selenium.NoSuchElementException: Unable to locate element: #login-email123
For documentation on this error, please visit: http://seleniumhq.org/exceptions/no_such_element.html
Build info: version: '3.14.0', revision: 'aaccccc0', time: '2018-08-02T20:13:22.693Z'
System info: host: 'DREAMBOX', ip: '192.168.137.1', os.name: 'Windows 10', os.arch: 'amd64', os.version: '10.0', java.ve
Driver info: org.openqa.selenium.firefox.FirefoxDriver
Capabilities {acceptInsecureCerts: true, browserName: firefox, browserVersion: 72.0.2, javascriptEnabled: true, moz:acce
Session ID: 5fcc5c00-5e4e-4e22-b3ea-c6768cb5bdf7
*** Element info: {Using=css selector, value=#login-email123}
        at sun.reflect.NativeConstructorAccessorImpl.newInstance0(Native Method)
        at sun.reflect.NativeConstructorAccessorImpl.newInstance(NativeConstructorAccessorImpl.java:62)
        at sun.reflect.DelegatingConstructorAccessorImpl.newInstance(DelegatingConstructorAccessorImpl.java:45)
        at java.lang.reflect.Constructor.newInstance(Constructor.java:423)
        at org.openqa.selenium.remote.http.W3CHttpResponseCodec.createException(W3CHttpResponseCodec.java:187)
        at org.openqa.selenium.remote.http.W3CHttpResponseCodec.decode(W3CHttpResponseCodec.java:122)
        at org.openqa.selenium.remote.http.W3CHttpResponseCodec.decode(W3CHttpResponseCodec.java:49)
        at org.openqa.selenium.remote.HttpCommandExecutor.execute(HttpCommandExecutor.java:158)
        at org.openqa.selenium.remote.service.DriverCommandExecutor.execute(DriverCommandExecutor.java:83)
        at org.openqa.selenium.remote.RemoteWebDriver.execute(RemoteWebDriver.java:548)
        at org.openqa.selenium.remote.RemoteWebDriver.findElement(RemoteWebDriver.java:322)
        at org.openqa.selenium.remote.RemoteWebDriver.findElementByCssSelector(RemoteWebDriver.java:416)
        at org.openqa.selenium.By$ByCssSelector.findElement(By.java:431)
        at org.openqa.selenium.remote.RemoteWebDriver.findElement(RemoteWebDriver.java:314)
        at com.automation.scripts.DynamicLoginExample.main(DynamicLoginExample.java:35)
Picked up _JAVA_OPTIONS: -Djava.net.preferIPv4Stack=true
```

Sub-Interfaces of WebDriver



- **WebDriver Interface** has **5** nested interfaces:
  - **Options Interface** - **Webdriver manage()** method returns Options interface which is required for managing different stuff in a browser like deleting cookies, managing options etc.
  - **Timeouts Interface** - **Options interface timeouts()** method returns Timeouts interface, required for implementing implicit wait and script or page timeout.
  - **Window Interface** - **Options interface window()** method returns Window interface, required for setting size and position of current window.
  - **Navigation Interface** - **Webdriver navigate()** method returns Navigation interface, required for doing browser back, forward, to and refresh.
  - **TargetLocator Interface** - **Webdriver switchTo()** method returns TargetLocator interface, required for switching to window, frame or alert.

Now let us implement implicit wait in parts:

```
//This method returns Options Interface
Options opt =driver.manage();
//Options Interface timeouts() method returns Timeouts Interface
Timeouts to = opt.timeouts();
//using Timeouts Interface implicitlyWait() method
to.implicitlyWait(30, TimeUnit.SECONDS);
```

Explicit Wait

- The **Explicit Wait** is used to tell the Web Driver to wait for certain conditions (**Expected Conditions**)
- The **explicit wait** is an intelligent kind of wait and applied on a **particular element**
- **Explicit wait** is implemented with **WebDriverWait** class which extends **FluentWait** class
- **WebDriverWait** class constructor takes two arguments:
  - The **WebDriver instance,** to pass to the expected conditions
  - The **timeout in seconds,** when an expectation is called
- **Explicit Wait** will ignore instances of **Not Found Exception** that are encountered on webpage and wait for certain conditions provided in the **until()** method
- **FluentWait** and **WebDriverWait** both are explicit wait and are implementations of **Wait interface**
- There is not much difference between **FluentWait** and **WebDriverWait**
- Since **WebDriverWait** extends **FluentWait**, the subclass **WebDriverWait** is used more often than **FluentWait**

```
Wait<WebDriver> wait2 = new FluentWait<WebDriver>(driver)
        .withTimeout(Duration.ofSeconds(30))
        .pollingEvery(Duration.ofSeconds(5))
        .ignoring(NoSuchElementException.class);

Wait<WebDriver> wait3 = new WebDriverWait(driver, 10)
.pollingEvery(Duration.ofSeconds(5))
.withTimeout(Duration.ofSeconds(30))
.ignoring(NoSuchElementException.class);
```

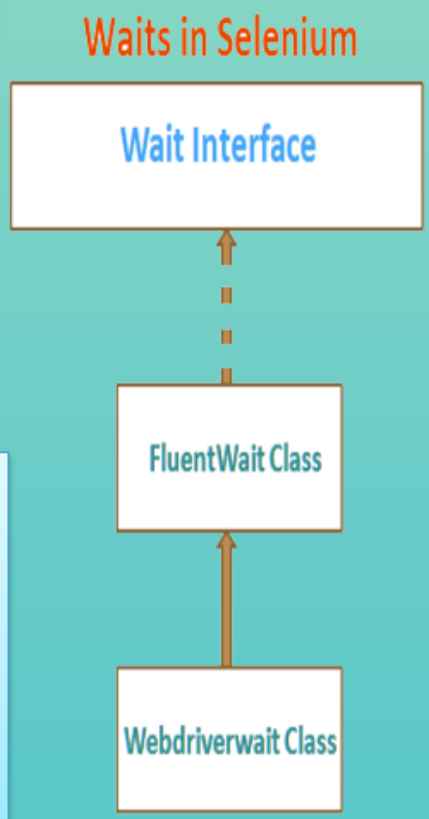
“until()” method in Fluent Wait

- **Explicit wait’s** condition is provided with “**until()**” method of **FluentWait class** and hence used with **WebDriverWait class.**
- This method takes the **predicate** as an argument which is a **clause** for performing the operation.

- When used with **WebDriverWait** object:
  - This method will **wait until** the condition provided returns neither **null or false**
  - If the condition is null or false and **timeout duration also expires**, this method will throw “**org.openqa.selenium.TimeoutException**”
  - In the timeout duration, this method will **poll(find)** the **WebElement** with the frequency of **500ms** (default)
  - As **1 second** has **1000ms** and if the timeout duration is of **30 seconds**, this method can **poll** for at most **60 times** before throwing **Timeout exception**.

```
WebDriverWait wait = new WebDriverWait(driver, 30);
WebElement submitEmail = wait.until(ExpectedConditions.presenceOfElementLocated(By.cssSelector("#login-submit")));
```

- This means using this method will wait for the **timeout duration** to complete before throwing the exception, else
  - It will continue polling for the element after every **500ms(which means 2 polls per second)**
  - If the condition is fulfilled within timeout duration, say after **10 polls which are equal to 5 seconds**, then method will return **expected return type** and flow will go to next statement in the program.



ExpectedConditions Class

- **WebDriver API** provides **ExpectedConditions class** which provides various **expectations or conditions** which can be used as a **dynamic waiting strategy** with **WebDriverWait** class
- **ExpectedConditions** class resolves into **predicate(condition)** which is expected to be **true**
- This class **provides the condition** which tries finding the web element, depending on its status
- **ExpectedConditions** class has number of methods to wait dynamically and which returns either **Boolean** or **WebElement:**
  - **ExpectedCondition<WebElement>** - This is the type of condition which has a web element locator as parameter If the condition can find the element, it returns the **WebElement** as result
  - **ExpectedCondition<Boolean>** - This type of condition has a string parameter. If the result of applying the condition is true**, true** is returned as result
- Every method of **ExpectedConditions class** returns condition as object of **ExpectedCondition interface**
- All methods of **ExpectedConditions class** are **static**, so they can be used directly with class name

ExpectedCondition < WebElement > **elementToBeClickable**(By locator)

ExpectedCondition < Boolean > **elementToBeSelected**(By locator)

ExpectedCondition < WebElement > **presenceOfElementLocated**(By locator)

ExpectedCondition < Boolean > **titleContains**(String title)

ExpectedCondition < Boolean > **titleIs**(String title)

ExpectedCondition < Boolean > **urlContains**(String fraction)

ExpectedCondition < Boolean > **urlToBe**(String url)

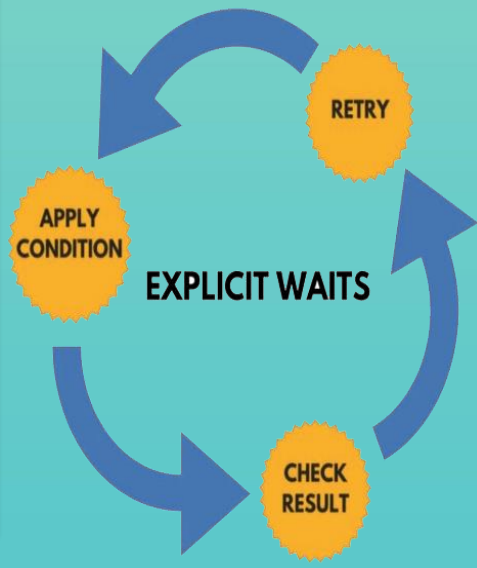
ExpectedCondition < WebElement > **visibilityOfElementLocated**(By locator)

Explicit Methods Returning WebElement

- **ExpectedConditions class** methods which return **WebElement** are useful to locate element when webpage contains lot of **AJAX**
  - **ExpectedCondition<WebElement> presenceOfElementLocated(By locator)** - An expectation for checking that an element is present on the DOM of a page.
  - **ExpectedCondition<WebElement> visibilityOfElementLocated(By locator)** - An expectation for checking that an element is present on the DOM of a page and visible. Visibility means that the element is not only displayed but also has a height and width that is greater than 0.
  - **ExpectedCondition<WebElement> elementToBeClickable(By locator)-** An expectation for checking an element is visible and enabled such that you can click it.

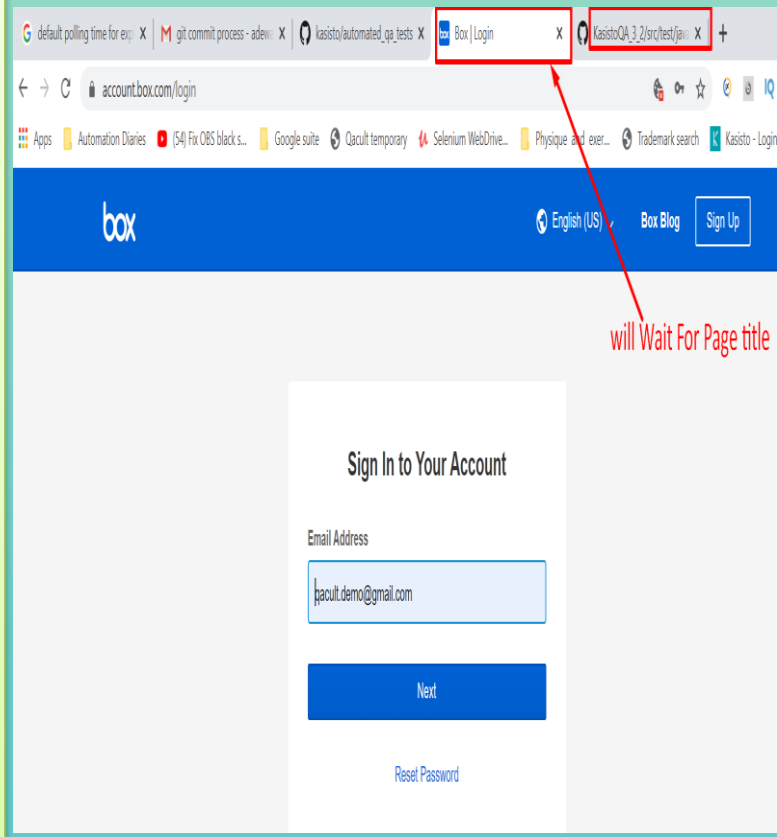
- All these 3 methods can be used to find **WebElement** and practically they have no priority.
- **90% WebElement** can be found with **presenceOfElementLocated()** method and other two methods are used rarely.
- **ExpectedConditions** class's **elementToBeClickable(By locator)** method has a overloaded form which takes **WebElement** as argument.
  - **ExpectedCondition<WebElement> elementToBeClickable(WebElement element)**

Please note: There are also some other important methods of **ExpectedConditions class** which uses different expectations and may have return type as **Boolean** or **List<WebElement>**. These methods will be covered later.



Verify Page Title Explicitly

- **ExpectedConditions class** provides <u>two expectations</u> which can be used to wait for **web page title** dynamically:
  - **ExpectedCondition<Boolean> titles(String title) -** An expectation for checking the title of a page which should be **exact match** of page title.
  - **ExpectedCondition<Boolean> titleContains(String title) -** An expectation for checking that the title contains a case-sensitive **substring or part** of page title.
- Both these method:
  - Return **Boolean** depending upon condition is **True**.
  - Will wait for **String argument** to match the page title during **timeout** duration.
  - Will throw **Timeout exception** if page title does not match within **timeout** duration.
- In automation, it's a good practice to verify the page **title** whenever we navigate to new webpage.
- "**titleContains(String title)**" is more preferred than "**titles(String title)**" because page title may contain leading and trailing spaces.



Quiz

- How many types of waits are applicable in **WebDrive**r scripts?
- What is **static wait** and why it is not recommended to use?
- How can you implement **implicit wait** in **WebDriver** scripts?
- What is the meaning of **Implicit wait**?
- When does WebDriver throw **NoSuchElementException?**
- Can you please name **5 sub-interfaces** of **WebDriver**?
- What is **Explicit Wait**?
- List two differences between **Implicit and Explicit Wait**
- Please explain how "**until()**" method of **FluentWait** class works?
- How will you **verify title** of webpage **explicitly**?

Homework

- Create **Login and Logout WebDriver script** on "**http://app.box.com**" and:
  - Use Implicit Wait on all **WebElements**.
  - Find each **WebElement** using Explicit wait.
  - Please verify the **title of page** at every new page.

- Create a **new folder script**:
  - After login to box app create a new folder dynamically.
  - While creating folder, **select permission** as "**Viewer**"
  - Verify the **success message** that appears at the top of the page.